Paper 01 — Quotes and Design Notes

Working notes. Copy-paste fodder for main.typ.

"LLMs strip provenance from knowledge. Systematically, architecturally and by design. And in so doing, AI systems are creating a form of knowledge network decay that degrades the knowledge infrastructures that human civilization rely upon." [1]

"The concept, first formalized by Giorgio Cencetti in 1939, captures the essence of the critical importance of provenance: knowledge does not exist as isolated units. It exists in structured relationships, and those relationships are themselves carriers of meaning. Destroy the relationships and intelligibility is lost." [1]

"The historian Peter Burke, in *What is the History of Knowledge?* (2016), situates this principle within a broader intellectual context. Burke argues that to qualify as 'knowledge,' items of information must be discovered, analyzed, and systematized — what he calls, *Verwissenschaftlichen*, the shift towards a more scientific approach. Burke posits that knowledge is not raw data. It is information that has been processed through systems of verification, classification, and contextual placement. The mechanisms of that processing — and their history — are themselves a form of knowledge. Burke further argues that even the idea of scientific objectivity — 'an attempt to separate knowledge from the knower' — has a history. Provenance is how we preserve the record of that processing. Without it, we collapse knowledge back into unverified assertion." [1]

"The quest for knowledge rather than mere information is the crux of the study of archives… All the key words applied to archival records — provenance, respect des fonds, context, evolution, interrelationships, order — imply a sense of understanding, of 'knowledge,' rather than the merely efficient retrieval of names, dates, subjects, or whatever, all devoid of context, that is 'information.'" — Council on Library and Information Resources, via [1]

"Suzanne Briet, librarian, historian and poet, in her 1951 manifesto *What Is Documentation?*, argued that a document is not simply a text — it is any piece of evidence organized to represent or prove something. An antelope in the wild is not a document. An antelope cataloged in a zoo, with a record of its capture, its species classification, its provenance of origin — that is a document. The act of documentation is what transforms raw existence into knowledge that can be evaluated, transmitted, and trusted. Without the record, there is no evidence. Without evidence, there is no knowledge — only assertion." — via [1]

"Patrick Wilson, writing on cognitive authority in *Second-Hand Knowledge* (1983), made a complementary point: we accept most of what we know not through direct experience but on the authority of others. The question is never simply what is claimed but who claims it, on what basis and the source of the claim. Wilson argued that the credibility of knowledge relies upon our ability to trace the lineage of information or a claim. And tracing works to their sources requires infrastructure and knowledge." — via [1]

"Subject headings and classification systems organize knowledge into retrievable, navigable structures so that a researcher can move from a question to a source, from a source to its author and to that author's citations. Every layer of this system is provenance infrastructure — designed to make knowledge findable and evaluable. In library science, provenance is documentation. It is the chain of custody that connects a claim to its source, a source to its author, and an author to the context in which they produced knowledge. This chain is the mechanism by which knowledge becomes trustworthy. Large language models break this chain by design." [1]

Note: RankeDB's positioning — rebuild the provenance chain LLMs break, using LLMs as workers, for LLMs as consumers. The tool that severed provenance becomes the tool that restores it, inside an architecture that refuses to let it strip the chain.

"Science, industry, and society are being revolutionized by radical new capabilities for information sharing, distributed computation, and collaboration offered by the World Wide Web. This revolution promises dramatic benefits but also poses serious risks due to the fluid nature of digital information. One important cross-cutting issue is managing and recording provenance, or metadata about the origin, context, or history of data." — Cheney, Chong, Foster, Seltzer & Vansummeren (OOPSLA 2009), via [1]

Note: Provenance was identified as a cross-cutting research priority in computer science two decades before LLMs broke it at scale. The irony: the discipline had the theoretical groundwork ready when the problem arrived, but the architectural integration — provenance as substrate rather than annotation — was never built. RankeDB picks up that thread.

Scope note (important — must be stated explicitly in Part 1):

RankeDB does not aim to solve global provenance. It targets personal up to small-enterprise scale: individual archives, project teams, small organizations. At this scale, the sources are largely self-trusted — your own email, your own chats, your bank statements, your photos, your team's documents. The trust problem is about preserving the chain from things you already trust, not establishing trust across an adversarial public.

Where RankeDB does NOT go: Wikipedia-scale consensus, web-scale retrieval, public scientific record. Those are different problems with different failure modes (adversarial editors, commercial stakes, peer review, citation networks). Trying to solve them all at once is what killed the Semantic Web.

Why this matters architecturally: at personal/project scale, the auto-verifiable domain is usually close to the whole domain, and the bounded-verification model works. At global scale the unverifiable case dominates, and a different architecture is needed. RankeDB is tuned for the former and honest about it.

Design notes for paper 1, Part 1: two principles, one stance

Provenance and consensus are orthogonal problems

Knowledge graphs have been trying to solve two different problems with one system: provenance (who said what, traceable back to sources) and consensus (what should we agree is true). These are orthogonal.

- **Provenance** is an *attribution* problem: who said what, when, on what basis, derived from what. Solvable by construction — just do not throw the chain away. RankeDB is a provenance database.
- **Consensus (common truth)** is a *social* problem: getting multiple observers to agree on what to trust. Requires voting, authority hierarchies, negotiation, peer review, time. Wikipedia solves it at scale with enormous human effort. The Semantic Web tried to pre-solve it via global ontology and failed.
- **Absolute truth** is philosophically incoherent — no one can have it, so drop it from the design.

The Semantic Web's failure is not that its ontology was too ambitious. It is that it tried to pre-bake consensus into the substrate, which was a category error. RankeDB separates the two: provenance is handled rigorously as a database problem, consensus is left to the layer above, where humans, applications, and time can work on it. This separation is the core contribution.

RankeDB stores *attributed claims*. Every node in the graph is a communicative act by someone, at some time, in some context. "Napoleon was born in 1769" is not a fact in RankeDB — it is Wikipedia's claim,

or your history textbook's claim, or your grandfather's claim. The graph stores the claim, the claimer, the context, and the provenance. It does not store "the truth" about Napoleon. What consumers do with the claims is their business.

Consequences:

- **Contradiction is normal, not a bug.** Two nodes can claim opposite things; both stay in the graph; both carry their provenance.
- **Conviction replaces certainty.** A claim is not "true" or "false" — it has a conviction score based on corroborating sources, their authority, and who is asking.
- **The same claim can mean different things** depending on who said it, when, and to whom. Context is preserved, not abstracted away.
- **There is no "ground truth" layer.** Level 0 is the archive of communicative acts, not an archive of how the world is.
- **Ontology emerges per-perspective**, not globally. My understanding of who "Bob" is may differ from yours, and that is fine — the graph holds both.

Consensus workers, if an application wants them, are implemented on top of the substrate: they produce `classification/consensus` or `observation/consensus` nodes that aggregate views with their own provenance. But the database itself makes no consensus decisions. *We do not refuse common truth, we defer it to the consumers and provide the substrate that makes it possible.* Consumers who want consensus can build it. Consumers who want to preserve dissent can preserve it. Consumers who want to pick a single perspective can do that too.

This is the constructive stance. RankeDB is not rejecting truth — it is identifying a conflation the field has been stuck on for 30 years and fixing it. Consensus is downstream of provenance, not part of it. You cannot have meaningful consensus without attribution, but you *can* have meaningful attribution without consensus. One is the foundation; the other is a choice built on top.

Bounded scope: personal to small-enterprise

The separation of provenance from consensus becomes tractable at personal up to small-enterprise scale: individual archives, project teams, small organizations. At this scale, the provenance problem is solvable *and* the consensus problem is small enough to defer comfortably.

- **Consensus is not needed** for most questions. I do not need to agree with anyone else about what my mother said in her email. I just need to preserve what she said.
- **Ontology is bounded.** The entities that matter in my life are finite. Resolving "who is Bob" across 200 conversations is tractable. Resolving "who is Bob" across all humans named Bob on Earth is not.
- **Trust is pre-established.** I already trust my own sources. The question is not "is this source trustworthy?" but "did I capture what it said faithfully?"
- **Adversarial resistance shrinks.** My archive is mine. The threat model is "do not lose it, do not corrupt it," not "prevent millions of attackers from poisoning consensus."
- **Context is preserved by proximity.** All the documents that matter to me are about me, my work, my circle. Context stays intact because the scope stays intact.

Where RankeDB does *not* go: Wikipedia-scale consensus, web-scale retrieval, public scientific record. Those are different problems with different failure modes (adversarial editors, commercial stakes, peer review, citation networks). Trying to solve them is what killed the Semantic Web.

"RankeDB does not scale to Wikipedia" is not a weakness — it is a deliberate scope. Wikipedia-scale is the wrong target. The right target is the scale where the problem is solvable *and* the solution is actually useful to an individual. A knowledge graph about my life and work is more valuable to me than Wikipedia, because it is mine, it is complete, and it preserves context that global systems strip.

The two principles enable each other

The separation of provenance from consensus and the bounded scope are not two separate design decisions — they are one coherent stance.

- You can only afford to defer consensus if you have bounded the scale to one where the consumers who build on top can actually handle it. At global scale you would drown in contradictions with no human process to resolve them.
- You cannot justify bounded scope without the provenance/consensus separation — if you believed a single global truth layer was the goal, you would have to aim for global, because partial truth is incoherent.

Each enables the other. Together they define what RankeDB is:

RankeDB stores attributed claims; common truth is what consumers build on top when they want it.

This is paper 1′s thesis in one sentence. The rest of the paper — the three levels, the taxonomy, the invariants, the rebuild guarantee, the under-prescription principle — all fall out as the structural consequences of these two commitments.

Note: RankeDB aligns with Wilson's cognitive authority framing and Briet's documentation thesis: the database does not care about the antelope itself, only about who said what about the antelope, when, and on what basis. This is a deliberate departure from Berners-Lee's Semantic Web vision, which tried to ground meaning in global concept definitions. RankeDB treats the communicative act as primary.

Comparison: Karpathy's LLM Wiki (April 2026)

Karpathy's LLM Wiki (gist 442a6bf, 5000+ stars) proposes a three-layer pattern: immutable raw sources, an LLM-maintained wiki of markdown files, and a schema document. The LLM incrementally builds and maintains entity pages, summaries, cross-references, and contradiction flags. The core insight — compounding knowledge vs. stateless RAG — is the same as RankeDB's.

What it validates: the need for persistent, structured, LLM-maintained knowledge that compounds over time rather than being re-derived on every query. Karpathy's "LLM does bookkeeping, human does thinking" is RankeDB's worker model. His immutable raw sources are L0. His wiki is L1+L2 collapsed into flat markdown.

What it's missing — and what the comments are already discovering:

- **No provenance.** Wiki pages are updated in place. You cannot trace a claim back through its derivation chain to the source that produced it. Karpathy's log.md records *when* things were ingested, not *how* a claim was derived.
- **Destructive consolidation.** Entity pages get overwritten. When the LLM "updates" a page with new information, the old synthesis is silently replaced. The contradiction Karpathy says the system "flags" gets resolved at write time by the LLM, with no record of what was there before. Git history is the only safety net.
- **No content type taxonomy.** Everything is "a markdown page." No distinction between fact, summary, classification, observation. No way for downstream consumers to filter by derivation type.
- **Schema as instruction, not architecture.** Karpathy relies on a CLAUDE.md file telling the LLM how to behave. RankeDB's invariants are enforced by the API — the system *refuses* to create a node without provenance, regardless of what the LLM wants to do.
- **The comments prove the gap.** Within days, users are independently reinventing RankeDB's invariants ad-hoc: access control via capability tokens, contamination firewalls, verify-before-assert hooks, file locking, contradiction callouts with claim types (source/analysis/unverified/gap). One

commenter (redmizt) built 13 architectural extensions on top of flat files to solve problems that RankeDB's three invariants (immutability, acyclicity, mandatory provenance) prevent by construction.

The key failure mode: Karpathy writes "noting where new data contradicts old claims" — but his wiki overwrites the old claims when it updates entity pages. This is exactly the destructive consolidation §3.2 argues against. The LLM quietly resolves contradictions at write time, with no provenance, no record of the prior belief, and no way to recover if the resolution was wrong.

Positioning for paper 1: Karpathy's LLM Wiki validates the need for the pattern RankeDB proposes while demonstrating exactly the failure modes that RankeDB's invariants prevent. It is the "vibe wiki" — compelling at small scale with an engaged human, but structurally unable to preserve the derivation history that makes knowledge trustworthy. RankeDB is what you get when you take the same insight and refuse to cut corners on provenance.

Design note: buggy workers, blast radius, and purging

When a worker has a bug, the provenance chain gives you the complete blast radius instantly: every node produced by that worker run (identified by run ID), and every node derived from those nodes, all the way down the DAG. No guessing, no auditing, no "which pages did the LLM touch last Tuesday?"

Two responses:

- **Non-destructive invalidation.** Mark the run's outputs as invalidated. They stay in the graph (immutability preserved) but consumers filter them out. Downstream nodes that depended on them are automatically suspect — their provenance chain passes through invalidated nodes. A replacement worker re-runs on the same sources and produces new nodes alongside the old ones.
- **Destructive purge.** Administrative operation outside the knowledge model. Creates a cropped copy of the graph with the affected branch removed. The original graph should be backed up before purging — the purge is irreversible by design, and the backup preserves the full history for forensic or developmental purposes.

Purging is especially valuable during development: run an experimental worker, inspect the results, purge if they are bad, re-run with improvements. The sources are untouched. The fix is an append on a clean graph, not a migration on a corrupted one.

This is the concrete operational advantage of provenance + immutability
1. worker run IDs over the "just use git" approach. In a flat-file

wiki, a buggy LLM run silently corrupts pages and you have no way to know which pages were touched or what downstream conclusions were built on them. In RankeDB, the DAG *is* the audit trail. The rebuild guarantee (§3.5) means that after purging, the improved worker re-runs on the same L0 sources and the affected branch regenerates cleanly.

Strengthens §3.2 (immutability), §3.5 (rebuild guarantee), §5 (workers), and §7.2 (reprocessing without migration). Also a strong contrast point with Karpathy's LLM Wiki — where the equivalent operation is "revert the whole git repo and hope you find the right commit."

Implementation note: storage distribution

L0 and L1 are both in Postgres as *nodes with metadata and edges*. Only the raw blobs of source artifacts go to S3. The split is:

- **Postgres** = the graph (all nodes + all edges, L0 and L1). Content_type, encoding, origin, parent, provenance — everything that makes the graph queryable.

- **S3** = blob store (raw bytes only, keyed by SHA-256 hash). Referenced by L0 source nodes but not part of graph traversal.
- **FalkorDB** = the semantic index (L2 projection).

This means "one graph, three regions" is correct architecturally, not just philosophically: the regions span storage boundaries but the graph is one thing. Any node in Postgres can point to its blob in S3 (if it is a source) or to other nodes (if it is derived). The API treats it all as one data structure.

The L0 tree invariant ("each source node has at most one parent") is enforced in Postgres, where the edges actually live. S3 does not need to know about the tree — it is just a content-addressed blob store.

Update §4.1 during next pass to reflect this cleaner mental model:

- **§4.1.1** should describe S3 as *the blob store*, not as Level 0. Blobs referenced by hash.
- **§4.1.2** should describe Postgres as holding *the entire graph structure* — L0 source nodes with blob references, L1 derived nodes, and all provenance edges between them. The node authority for the whole DAG, not just Level 1.
- **§4.1.3** FalkorDB remains the L2 semantic graph projection.

The storage split is about *byte payload* (S3 for blobs, Postgres for metadata and structure), not about *logical level*. L0 and L1 share the same Postgres tables because they share the same graph.

Implementation note: dark storage + content-addressed blob pool

Refinement of the storage distribution. S3 is pure content-addressed dark storage — nobody lists it, nobody browses it. The API is the only door. All access patterns reduce to GET/PUT/HEAD by hash.

Ingest is to the API, always. Workers never touch S3 or Postgres directly. The API:

1. Computes the hash of the incoming blob.
2. Idempotently PUTs to S3 (no-op if hash is already there).
3. Creates a Postgres node with metadata and the hash pointing at the S3 blob.

Workers don't need S3 credentials. Workers don't know S3 exists. That is what "dark storage" means — it is hidden backend infrastructure, not part of any ACL surface.

Defining invariant: *if it is not in Postgres, it is not there.* Postgres is the complete index of what exists. An orphaned S3 blob (somehow present without a Postgres node) is unreachable through the API. Content-addressed storage means orphans are benign — they take space but can't be referenced. GC is optional: a Postgres query that finds hashes in S3 not referenced by any node.

L1 blobs go to S3 too. Summaries, normalized conversations, OCR text, transcripts, extracted fact text — all are content blobs. They get the same treatment as L0 blobs: content-addressed in S3, referenced by hash in Postgres. This keeps Postgres slim.

The rule: Postgres = graph topology + metadata + indices. S3 = content.

What stays in Postgres:
- Graph structure (nodes + edges)
- Small queryable metadata (content_type, encoding, origin, timestamps, parent, tags, conviction scores, validity windows)
- Full-text search indices (tsvector; see caveat below)
- Vector embeddings (pgvector — small, derived)

What goes to S3:
- L0 source bytes (original artifacts)
- L1 normalized content (conversation text, OCR, transcripts)

- L1 derived content (summaries, fact text, entity descriptions)

Caching at the API level. Content-addressed means cache invalidation is impossible by construction — content at hash X is content at hash X, forever. Pure LRU. Three-tier: API process memory → local disk → S3. Reloads are deterministic, eviction is safe. Hot working set is typically small.

Caveats:
- Full-text search wants text in the same row as the tsvector. Probably store extracted text in Postgres for indexing *and* in S3 as canonical blob. tsvector plus plain text is cheap; it keeps text search self-contained.
- Vector embeddings are tiny, stay in Postgres with pgvector.

Consequences:
- Postgres rows become uniform: metadata + hash, a few hundred bytes each. The graph fits in RAM at personal/project scale.
- Backup asymmetry: Postgres is small and frequent; S3 is bulky and occasional.
- Forking Postgres stays cheap — one S3 blob pool, many graph instances. Experiments, A/B tests, development branches, all share the same content.
- Storage abstraction is minimal — S3, R2, Backblaze, IPFS, local filesystem, anything that supports content-addressed GET/PUT/HEAD. Swapping backends is trivial because the interface is small.

Supersedes earlier thinking about ACL-split between ingest workers and backend (where ingest workers had S3 `PutObject`). Under dark storage, *no worker ever touches S3.* The API is the only S3 client.

Implementation note: Postgres as tunable cache over S3

Refinement: content duplication in Postgres is not an architectural commitment but a runtime cache policy. The threshold (what size of blob gets cached inline) is a config knob that can be raised or lowered at runtime without data loss.

Every node carries fields related to content:
- `content_hash` — SHA-256 pointing at the blob in S3 (canonical storage)
- `content_size` — size in bytes, for policy queries
- `encoding` — format identifier; used to determine cache eligibility (see below)
- `content_cached` — `text` column holding inlined text content, NULL if not cached or not text

Encoding is MIME-style: `class/format`. Borrowed directly from HTTP's Content-Type convention. The top-level class indicates what kind of content it is; the subtype indicates the specific format:

```
text/eml          text/chatgpt       text/whatsapp
text/normalized   text/markdown      text/plain       text/json
image/png         image/jpeg         image/heic
audio/wav         audio/mp3
video/mp4
application/pdf   application/dkb-csv
```

Cache eligibility falls out of the naming scheme:

```
WHERE encoding LIKE 'text/%'
```

Anything with prefix `text/` is cacheable. Everything else is S3-only. No hardcoded enum, no lookup table, no `is_text_content` column. The policy is expressed in the naming convention. New text encodings (`text/rss`, `text/ical`) become cacheable automatically. New binary encodings (`image/avif`) stay in S3 automatically.

Bonus: the encoding field becomes self-describing for consumers. An API endpoint serving the blob can set the HTTP `Content-Type` header directly — no translation needed. A worker dispatching by content type knows the handler family from the class prefix.

The threshold is cache-fill policy. Raise from 8 KB to 32 KB: a background pass fills the cache for eligible (text) nodes.

```
SELECT id, content_hash, content_size FROM nodes
WHERE content_size <= 32768
  AND encoding LIKE 'text/%'
  AND content_cached IS NULL;
```

Lower from 32 KB to 8 KB: a background pass evicts.

```
SELECT id FROM nodes
WHERE content_size > 8192 AND content_cached IS NOT NULL;
```

Properties:

- Policy is queryable (SQL shows exactly what is cached).
- Policy is reversible (raise/lower threshold is just fill/evict; S3 always holds canonical content).
- Multiple policies can coexist (cache summaries always, cache conversations under 32 KB, never cache transcripts — each a WHERE clause).
- Per-node pinning is easy (a `pin` flag column). Hot content above threshold can be pinned.
- Partial cache is safe. API falls back to S3 on miss. No correctness dependency on cache state.
- Cost/performance tuning is live. Adjust threshold under memory or latency pressure. No downtime.

Schema suggestion: split the cache from the main nodes table. A lean `nodes` table with metadata + hash + size. A separate `node_content_cache` table keyed by hash. Two wins:

1. Cache operations don't cause page I/O or row locking on graph metadata.
2. Deduplication by hash — if multiple nodes reference the same blob (possible with content-addressing), they share a single cache entry.

Binary content is never in Postgres. A PNG stays in S3. Its text interpretation (OCR output) is a separate derived node with its own hash, its own `encoding: text`, its own cache row. The graph naturally distinguishes the binary source from its textual interpretation — both preserved with provenance. SQL never operates on binary bytes, only on their text derivations.

Summary: S3 is canonical, stores all content (text and binary). Postgres is a tunable cache over text content only, plus the graph structure itself. Cache eligibility is determined by encoding. Binary encodings never cache. Text encodings cache up to a runtime-tunable size threshold. Text search operates on cached text. Binary content access always goes through S3. Nothing is lost by changing the threshold because S3 is always the source of truth.

Bibliography

[1] J. Talisman, "Where Provenance Ends, Knowledge Decays."